AgentLighthouse — Hackathon Winning Playbook

Oct 20, 2026 · @TARNIJA

0. Read me first

Submission closes **today, 20 October 2026**, so we have roughly 8 working hours. Open the Unstop page now, confirm the exact cutoff time, and set an alarm 60 minutes before it. This playbook is ordered so a submittable version exists by **21:30 IST**, and everything after that is polish.

What we are building: AgentLighthouse, "Lighthouse for AI agents." You paste a website URL. TinyFish web agents attempt real tasks on it (find the price, find the return policy, reach checkout) and we return an Agent-Readiness Score from 0 to 100. Each result shows the exact step where the agent got stuck, an AI-written diagnosis, and copy-paste fixes.

How to use this playbook

1. Everyone reads sections 0 to 3 first (15 minutes).
2. Each person takes the phases listed against their role in section 3.
3. Whoever drives the coding agent (Claude Code, Cursor, Codex) pastes `design.md` (section 7), `agents.md` (section 8) and the agent rules (section 9) into the repo before writing any code.
4. **No phase starts until the previous phase's gate checklist is ticked.** Gates are at the end of each phase section.
5. Sections 10 to 18 are for the non-coding work: UI prompt, README, video, social post, deck, revenue, judge Q&A. Start them in parallel from 16:00.

Six golden rules (a judge's view)

1. **End-to-end before pretty.** One URL going in and one scored report coming out beats five half-built features.
2. **TinyFish on screen.** The live browser stream must be visible in the UI and the video. Judges score what they can see.
3. **Cache every TinyFish response to disk.** The demo must work even if the network or credits fail on stage.
4. **Commit every 30 to 45 minutes** with clear messages. "Visible commits" is part of the 20 GitHub points.

5. **Never claim what doesn't run.** If a feature is not working, it goes under "Roadmap" in the README, not "Features."
6. **Short code wins.** Judges open the repo for about 2 minutes. Small files with obvious names read as senior work.

Hour-by-hour plan (IST, today)

Time	Phase	Done when
15:00 – 15:30	Phase 0: setup	Repo public, TinyFish smoke test passes
15:30 – 16:45	Phase 1: TinyFish task runner	One task runs, is cached and normalised
16:45 – 17:30	Phase 2: machine-readability signals (Fetch + Search)	Signals JSON for any URL
17:30 – 18:45	Phase 3: score and AI diagnosis	Full report JSON for 3 sites
18:45 – 19:30	Phase 4: API with live streaming	UI-ready endpoints pass curl checks
19:30 – 21:00	Phase 5: UI	Demo flow works in the browser
21:00 – 21:30	Freeze and benchmark	Leaderboard of 8 to 10 sites generated
21:30 – 23:00	Phase 6: ship	README, video, post, Unstop submitted
23:00 – cutoff	Buffer	Nothing new is built

Docs, deck and revenue research (sections 11 to 16) run in parallel from 16:00 with whoever is not coding.

1. The project in one page

One-liner: Google Lighthouse tells you how your site performs for humans. AgentLighthouse tells you whether an AI agent can actually use it, and shows you exactly where it gets stuck.

The problem. AI agents now shop, compare and book on people's behalf. Bain estimates agents could handle 15 to 25% of US e-commerce, worth $300 to 500 billion, by 2030 (Bain). Most websites were built only for human eyes. Cookie walls, unlabeled icon buttons, CAPTCHAs, infinite scroll and login walls silently turn agents away. Site owners have no idea this is happening, because a failed agent leaves no complaint and no analytics trail.

What exists today, and our gap. Static checkers already exist: Cloudflare's isitagentready.com, Frase, Chat Thing and open-source auditors. They **read files** (robots.txt, llms.txt, structured data). They do not **watch an agent try to do a job**. A site can pass every file check and still be a dead end for an agent that needs to click "Add to cart." We are the behavioural test, the way a crash test differs from reading a car's spec sheet.

What AgentLighthouse does

1. **Reads the site** with TinyFish Fetch: homepage, robots.txt, llms.txt, sitemap. It records bot blocks, JavaScript-only pages and missing machine-readable files.
2. **Picks real tasks** for that kind of site. An LLM classifies the site (shop, SaaS, travel, services) and selects 4 tasks such as "find the price of a product" or "reach checkout without paying."
3. **Sends TinyFish web agents to do them**, live. The user watches the agent's browser in our UI while it works.
4. **Diagnoses every failure with AI.** An LLM reads the agent's step trace and classifies the blocker (cookie wall, CAPTCHA, unlabeled button, login wall and so on), names the root cause, and writes a copy-paste fix.
5. **Scores and benchmarks.** The output is a 0 to 100 Agent-Readiness Score, a letter grade, and the **Agent Tax**: what each task costs an AI agent on your site in steps and dollars, compared with competitors found through TinyFish Search.

Why it wins this hackathon

- **TinyFish is the product, not a plugin.** We use four TinyFish surfaces: Agent, Fetch, Search and the live browser stream. Remove TinyFish and nothing is left.
- **It is AI/ML through and through:** site classification, failure-trace diagnosis, blocker taxonomy, fix generation, and a benchmark across sites.
- **It's instantly demoable.** Paste a URL, watch an agent struggle live, get a score. Judges understand it in 10 seconds.
- **It has a sharp business.** Every e-commerce and SaaS company will need this, and agencies can resell it.
- **It flatters the sponsor.** It positions TinyFish as the standard test harness for the agentic web.

The "Agent Tax" hook (use it everywhere). TinyFish bills agent work per step, about $0.016 per step (TinyFish). So every extra step a confusing website forces is a measurable cost. "Buying a phone on Site A takes an agent 9 steps. On Site B it takes 31, or it fails." That one sentence sells the product.

2. Judging rubric mapped to deliverables

Every hour of work should move one of these seven scores. If a task does not appear in this table, it waits until after submission.

Criterion (points)	What judges check	Our deliverable that earns it	Owner
Working GitHub project (20)	Public repo, working code, clear README, visible commits	Public repo from minute one; 15+ meaningful commits; README with a GIF, a 3-command quick start and a demo mode that runs without keys	Backend lead
TinyFish integration (20)	Meaningful, working, relevant; explained in README and video	Agent API runs every task; Fetch reads the site; Search finds competitors; live browser stream embedded in the UI; a "How we use TinyFish" README section with a table and code links	Backend lead
Innovation and idea (20)	Original, creative problem and solution	Behavioural agent testing (not static file checks), the Agent Tax metric, the blocker taxonomy, and a public leaderboard of real sites	Whole team, pitch owner
Social media post (10)	Public post showing what it does, with video or GIF and repo link	LinkedIn post plus X thread with a 30-second clip of the live agent and the leaderboard image	Docs owner

Criterion (points)	What judges check	Our deliverable that earns it	Owner
Technique (10)	Quality of technical approach, architecture, execution	Clean pipeline, typed data models, cached TinyFish calls, concurrency control, unit tests on scoring, `design.md` and `agents.md` in the repo	ML lead
Impact and usefulness (10)	Real problem, real people, realistic	Benchmark of 8 to 10 real sites with concrete fixes; one named persona (a D2C store owner)	Pitch owner
Revenue model (10)	Clear price economics, market research, justified prices	Pricing tiers with unit economics built on TinyFish's per-step cost; market sizing with sources (section 15)	Docs owner

What separates winners from the rest (judge's notes)

- **The first 20 seconds of the video decide the score.** Show the live agent browser and the score card before any slides.
- **A judge clicks the repo for 2 minutes at most.** The README must answer "what, why, how to run, how TinyFish is used" above the fold.
- **Numbers beat adjectives.** "7 of 10 top Indian shopping sites failed the checkout task" is remembered. "Many sites are not agent friendly" is not.
- **Honesty scores.** A "Limitations" section and a "Roadmap" section signal maturity. Judges punish features that are claimed but don't work.
- **Commits tell a story.** Judges can tell a repo built in one final push from one built in steps. Commit at every phase gate.

3. Team roles and ownership

Each role owns specific phases and files. Two people must never edit the same file at the same time, because merge conflicts at 22:00 lose hackathons.

Role	Owns (phases)	Owns (files)	Also responsible for
A. Backend and TinyFish lead	0, 1, 4	`app/tinyfish.py`, `app/main.py`, `app/audit.py`, `app/cache.py`	TinyFish credits budget, API keys, the README's TinyFish section
B. ML and scoring lead	2, 3	`app/signals.py`, `app/tasks.py`, `app/score.py`, `app/diagnose.py`, `app/llm.py`, `tests/`	Task goal prompts, blocker taxonomy, benchmark run at 21:00
C. Frontend lead	5	`web/index.html`, `web/app.js`	The UI prompt (section 10), screenshots and the GIF
D. Pitch and docs lead	6	`README.md`, deck, video, social post	Sections 11 to 16; the final Unstop submission
E. QA and research	Gate checks for every phase	`scripts/benchmark.py`, `data/sites.txt`	Picks the 10 benchmark sites, revenue research, judge Q&A drills

If the team is smaller, merge roles like this:

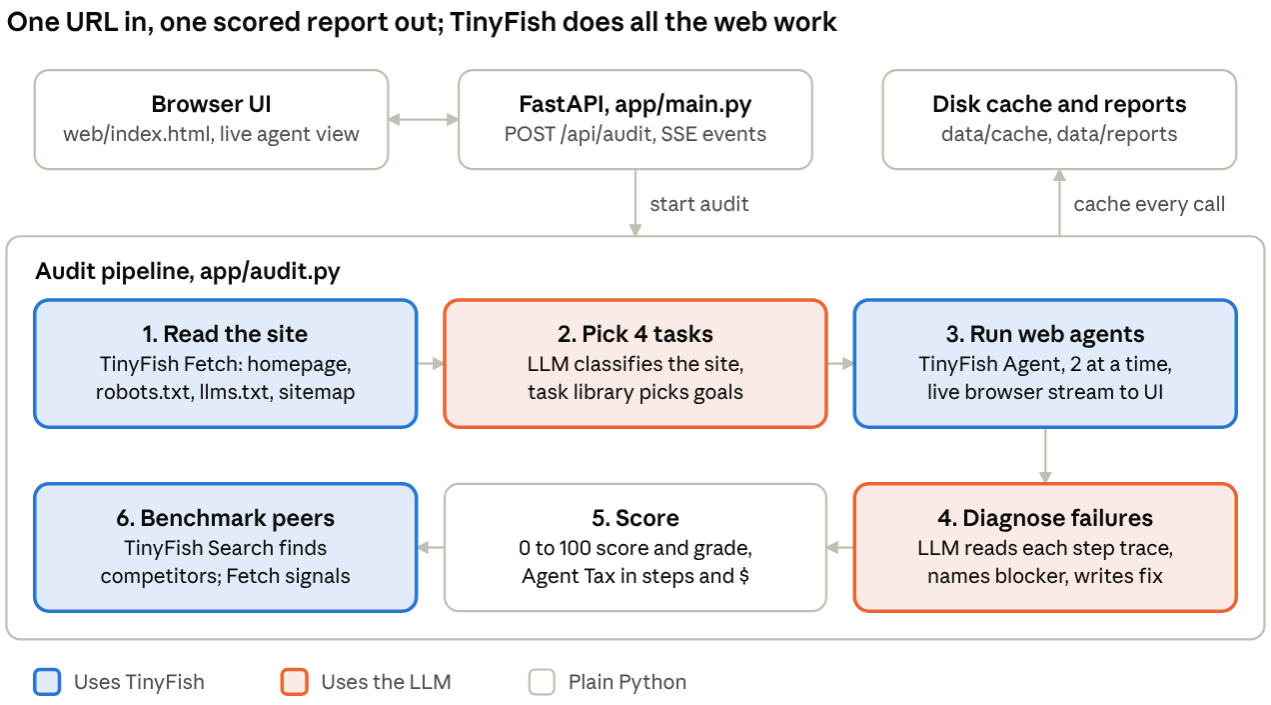
- **2 people:** Person 1 = A + B. Person 2 = C + D + E.
- **3 people:** Person 1 = A. Person 2 = B + E. Person 3 = C + D.
- **4 people:** A, B, C, and D + E merged.

Communication rules for today

1. One shared group chat. At each phase gate, the owner posts: `GATE Pn PASSED` plus the commit hash.
2. Blocked for more than 20 minutes? Post `BLOCKED: <what>` and switch to the fallback in section 17.
3. Only the backend lead merges to `main`. Everyone else works on a branch named after their role (`ml`, `ui`, `docs`).
4. API keys are shared through a private message, never committed. `.env` is in `.gitignore` from the first commit.

4. System architecture

The whole system is one Python service plus one HTML page. A URL goes in through the API, the audit pipeline runs six steps, and every TinyFish and LLM call is cached to disk so the demo can replay without network.



AgentLighthouse architecture · 6 pipeline steps, 3 on TinyFish

The UI never talks to TinyFish directly; it receives Server-Sent Events from our API, including each agent's live `streaming_url`, which it shows in an iframe.

Tech stack and why

Layer	Choice	Why this, today
Backend	Python 3.11, FastAPI, httpx, Pydantic	Fewest lines for an async API with SSE; typed models read well to judges
TinyFish	REST via httpx: Agent `run-async` plus `GET /v1/runs/{id}`, Fetch, Search	Explicit endpoints are easy to explain in the README and video. The official `tinyfish` Python SDK is an allowed swap
LLM	Any OpenAI-compatible model through the `openai` package (Claude, Gemini, Groq or OpenRouter)	Swap providers with two environment variables; JSON output mode

Layer	Choice	Why this, today
Frontend	One `index.html`, Tailwind via CDN, vanilla JS	No build step, nothing to break at 22:00
Storage	JSON files in `data/`	No database to set up; reports double as demo fixtures
Tests	pytest on scoring and signal parsing	Cheap proof of engineering quality
Hosting (optional)	Render or Railway free tier	Only if time remains after submission; a local demo is enough

Data contract (the one object everything passes around)

```
{
  "url": "https://example.com",
  "site_type": "ecommerce",
  "signals": {"fetchable": true, "llms_txt": false, "robots_ok": true,
"sitemap": true, "meta": true, "text_rendered": true},
  "tasks": [{"id": "find_price", "outcome": "success", "steps": 9, "seconds":
41, "blocker": "none", "answer": "₹1,299", "run_id": "..."}],
  "diagnoses": [{"task_id": "checkout", "blocker": "cookie_or_popup",
"root_cause": "...", "fix": "...", "severity": "high"}],
  "score": {"total": 62, "grade": "C", "success": 41, "efficiency": 9,
"readability": 12},
  "agent_tax": {"avg_steps": 17.5, "usd_per_task": 0.28},
  "peers": [{"domain": "competitor.com", "readability": 20}]
}
```

5. TinyFish integration spec

We use four TinyFish surfaces. Each one does a job nothing else in the stack can do, which is exactly what the 20 integration points reward. All details below were checked against the TinyFish docs today.

TinyFish surface	Endpoint	Where we use it	Why it matters to the product
Agent API (core)	`POST https://agent.tinyfish.ai/v1/automation/run-async`, then `GET /v1/runs/{id}`	`app/tinyfish.py` → `run_agent()`, step 3	The agent attempts each real task. Its success, step count and step trace *are* our measurement

TinyFish surface	Endpoint	Where we use it	Why it matters to the product
Live browser stream	`streaming_url` field on the run object	Sent to the UI over SSE and shown in an iframe	Users and judges watch the agent struggle live. This is the demo's hero moment
Fetch API	`POST https://api.fetch.tinyfish.ai`	`fetch()`, step 1 and step 6	Reads homepage, robots.txt, llms.txt, sitemap. Its error codes (`bot_blocked`, `empty_content`, `login_required`) are readiness signals in themselves
Search API	`GET https://api.search.tinyfish.ai`	`search()`, step 6	Finds 2 to 3 competitors so the score is relative, not abstract

Facts the code depends on (from the docs)

- Auth is one header, `X-API-Key`, read from the `TINYFISH_API_KEY` environment variable (docs).
- A run moves through `PENDING`, `RUNNING`, `COMPLETED`, `FAILED`, `CANCELLED`. **`COMPLETED` only means the browser worked, not that the goal succeeded,** so we always read our own `outcome` field from the result (runs).
- The run object carries `num_of_steps`, `streaming_url` (live view while running), `video_url` (recording, link expires in 15 minutes) and a `steps` array where each step has an `action` text, a `screenshot` and a `duration` (runs).
- `output_schema` forces the result into our JSON shape. It must be an object at the top level, `enum` values must be strings, and `additionalProperties`, `const` and `oneOf` are rejected (structured output).
- `agent_config.max_duration_seconds` caps a run's time for everyone. `max_steps` is beta-only, so we don't rely on it (endpoints).
- Pay-as-you-go accounts run **2 agents at once** (pricing). We use a semaphore of 2.
- Search is free up to 12,000 requests a day and Fetch up to 1,000 URLs a day (search, fetch). Only Agent steps spend our credits.

The whole client: `app/tinyfish.py`

```
"""Every TinyFish call lives in this file. Docs: https://docs.tinyfish.ai"""
import asyncio, json, os
import httpx
from app.cache import cached

HEAD = {"X-API-Key": os.getenv("TINYFISH_API_KEY", "")}  # empty is fine in
DEMO_MODE
AGENT = "https://agent.tinyfish.ai/v1"
DONE = {"COMPLETED", "FAILED", "CANCELLED"}


@cached
async def fetch(urls: list[str]) -> dict:
    """Clean page content plus per-URL error codes. Free tier: 1,000
URLs/day."""
    async with httpx.AsyncClient(timeout=150) as c:
        r = await c.post("https://api.fetch.tinyfish.ai", headers=HEAD,
                         json={"urls": urls, "format": "markdown", "links":
True})
        r.raise_for_status()
        return r.json()  # {"results": [...], "errors": [...]}


@cached
async def search(query: str, location: str = "IN") -> list[dict]:
    """Ranked web results: position, site_name, title, snippet, url."""
    async with httpx.AsyncClient(timeout=15) as c:
        r = await c.get("https://api.search.tinyfish.ai", headers=HEAD,
                        params={"query": query, "location": location})
        r.raise_for_status()
        return r.json()["results"]


@cached
async def run_agent(url: str, goal: str, schema: dict, on_update=None) ->
dict:
    """Start a web agent, poll until it ends, report progress to the UI."""
    body = {"url": url, "goal": goal, "output_schema": schema,
            "agent_config": {"max_duration_seconds": 150}}
    async with httpx.AsyncClient(timeout=30) as c:
        r = await c.post(f"{AGENT}/automation/run-async", headers=HEAD,
json=body)
        r.raise_for_status()
        run_id = r.json()["run_id"]
        while True:
            run = (await c.get(f"{AGENT}/runs/{run_id}", headers=HEAD,
```

```
                                params={"screenshots": "none"})).json()
            if on_update:
                await on_update(run)  # streaming_url + step count go to the
UI
            if run["status"] in DONE:
                return run
            await asyncio.sleep(3)


def parse_result(run: dict) -> dict:
    """COMPLETED is not success: read our own outcome field."""
    res = run.get("result") or {}
    if isinstance(res, str):
        res = json.loads(res) if res.strip().startswith("{") else {}
    if run["status"] != "COMPLETED":
        return {"outcome": "failure", "blocker": "other",
                "blocker_detail": (run.get("error") or {}).get("message")}
    return {"outcome": "failure", "blocker": "other", **res}  # defaults if
fields missing
```

The disk cache: `app/cache.py` (makes the demo network-proof)

```
import functools, hashlib, json, os, pathlib

DIR = pathlib.Path("data/cache")
DIR.mkdir(parents=True, exist_ok=True)


def cached(fn):
    """Save every result to disk; DEMO_MODE=1 replays only from disk."""
    @functools.wraps(fn)
    async def wrap(*args, **kw):
        plain = [a for a in args if not callable(a)]
        plain += sorted((k, v) for k, v in kw.items() if not callable(v))
        key = hashlib.sha1(json.dumps([fn.__name__, plain],
default=str).encode())
        path = DIR / f"{fn.__name__}-{key.hexdigest()[:16]}.json"
        if path.exists():
            return json.loads(path.read_text())
        if os.getenv("DEMO_MODE") == "1":
            raise RuntimeError(f"DEMO_MODE: nothing cached for
{fn.__name__}")
        out = await fn(*args, **kw)
```

```
        if not (isinstance(out, dict) and out.get("status") == "FAILED"):
            path.write_text(json.dumps(out))  # never cache infra failures
        return out
    return wrap
```

The task output schema (every agent run returns this shape)

```
BLOCKERS = ["none", "cookie_or_popup", "captcha_or_bot_block", "login_wall",
            "element_not_found", "unlabeled_control",
"slow_or_dynamic_content",
            "confusing_navigation", "other"]

SCHEMA = {
    "type": "object",
    "properties": {
        "outcome": {"type": "string", "enum": ["success", "partial",
"failure"]},
        "answer": {"type": "string", "nullable": True},
        "blocker": {"type": "string", "enum": BLOCKERS},
        "blocker_detail": {"type": "string", "nullable": True},
        "final_url": {"type": "string", "nullable": True},
    },
    "required": ["outcome", "blocker"],
}
```

Credit budget (keep it visible in the group chat)

TinyFish bills Agent work at about $0.016 per step, failed runs are free, and Search and Fetch cost nothing (TinyFish). The $13 starter credit therefore buys roughly 800 agent steps. Our estimate for one audit is 4 tasks at about 12 steps each, roughly 50 steps or $0.80. That gives about 15 full audits in total, so spend them deliberately:

Use	Audits	Approximate steps
Phase 1 smoke test and debugging	2	100
Phase 3 report checks	2	100
Benchmark leaderboard at 21:00	9	450
Video recording	1	50
Spare	1	100

If credits run low, top up the $10 minimum at the TinyFish wallet rather than opening extra accounts. **Never run an uncached audit twice for the same URL;** the cache makes reruns free.

Goal-writing rules for agent tasks (vague goals fail; the TinyFish docs say so explicitly)

1. State the role: "You are testing whether an AI agent can use this website."
2. State one concrete task with a clear finish line.
3. State the safety limits: no login, no personal data, no payment, no form submission.
4. Say what to do about obstacles: "If a popup appears, try to close it and continue."
5. Define `success`, `partial` and `failure` for this task in one sentence each.
6. Ask the agent to pick the single main blocker from the enum.

6. Repo structure and coding standards

The target is **under 900 lines of Python and under 500 lines of HTML and JS** in total. A judge should understand the whole backend by reading file names and the first docstring of each file.

```
agentlighthouse/
├── README.md            # what, why, how to run, how TinyFish is used
├── design.md            # the build contract (section 7)
├── agents.md            # log of everything AI wrote (section 8)
├── .env.example         # TINYFISH_API_KEY, LLM_BASE_URL, LLM_API_KEY,
LLM_MODEL, DEMO_MODE
├── requirements.txt     # fastapi uvicorn httpx pydantic openai pytest
python-dotenv
├── app/
|   ├── main.py          # FastAPI routes + SSE                      (~80
lines)
|   ├── audit.py         # runs the 6-step pipeline, emits events    (~110
lines)
|   ├── tinyfish.py      # every TinyFish call                       (~70
lines)
|   ├── runner.py        # one task -> one TaskResult, 2 at a time   (~45
lines)
|   ├── cache.py         # disk cache + DEMO_MODE replay             (~25
lines)
|   ├── signals.py       # machine-readability checks from Fetch     (~70
lines)
|   ├── tasks.py         # task library + goal prompts + schema      (~90
lines)
|   ├── llm.py           # one function: ask_json(prompt) -> dict    (~25
lines)
|   ├── diagnose.py      # failure trace -> blocker, cause, fix      (~60
lines)
|   ├── score.py         # score, grade, Agent Tax                   (~60
lines)
```

```
|   └── models.py        # Pydantic models for the data contract      (~60
lines)
├── web/
|   ├── index.html       # the whole UI (Tailwind CDN)
|   └── app.js           # SSE client + rendering
├── scripts/
|   ├── smoke_tinyfish.py  # Phase 0 check: one Fetch, one Search, one tiny
agent run
|   └── benchmark.py       # audits data/sites.txt -> data/leaderboard.json
├── data/
|   ├── sites.txt        # the 10 benchmark URLs
|   ├── cache/           # committed! lets anyone run DEMO_MODE=1
|   └── reports/         # one JSON report per audited domain
└── tests/
    ├── test_score.py
    └── test_signals.py
```

Coding standards (paste these into the coding agent too)

1. **Functions over classes.** Use Pydantic models only for data; logic lives in plain functions.
2. **Each function under 30 lines; each file under 150 lines.** If a file grows past that, split it by job, not by type.
3. **Every file starts with a one-line docstring** saying its single job.
4. **Type hints on every function signature.**
5. **Names say what things are:** `run_agent`, `parse_result`, `score_report`. No `utils.py`, `helpers.py` or `manager.py`.
6. **No dead code, no commented-out code, no print debugging** left in `main`.
7. **Configuration only through environment variables,** loaded once with `python-dotenv` in `main.py`.
8. **Errors:** catch at the edges (`audit.py` and `main.py`), turn them into a UI event, and keep the audit going. One failed task must never crash an audit.
9. **Formatting:** `ruff format` before every commit. Line length 100.
10. **Commit messages:** `phase N: <what works now>`, for example `phase 1: agent task runs, cached, parsed`.

`app/llm.py`, the only LLM code in the repo

```
"""One way to call the LLM: a prompt in, a JSON dict out."""
import json, os
from openai import AsyncOpenAI
from app.cache import cached

client = AsyncOpenAI(base_url=os.getenv("LLM_BASE_URL"),
api_key=os.getenv("LLM_API_KEY", "x"))


@cached
async def ask_json(system: str, user: str) -> dict:
    r = await client.chat.completions.create(
        model=os.environ["LLM_MODEL"],
        response_format={"type": "json_object"},
        messages=[{"role": "system", "content": system},
                  {"role": "user", "content": user}],
        temperature=0,
    )
    return json.loads(r.choices[0].message.content)
```

The `openai` package works with any OpenAI-compatible endpoint, so the team can use whichever provider it has a key for by changing `LLM_BASE_URL` and `LLM_MODEL`. Confirm your provider supports JSON output mode in the Phase 0 smoke test.

7. design.md (paste into the repo root)

This file is the build contract. The coding agent reads it before every phase, and any change to scope is made here first, then in code. Copy the block below as-is.

```
# AgentLighthouse: design.md

## 1. Product
AgentLighthouse audits how well AI web agents can use a website. Input: one
URL.
Output: a report with an Agent-Readiness Score (0-100), grade, Agent Tax,
per-task results, AI diagnoses with fixes, and a comparison with 2-3
competitors.

## 2. Users
- Primary: D2C / e-commerce owners and their dev agencies.
- Secondary: SaaS growth teams, SEO/GEO agencies, platform teams.

## 3. Scope
IN: single-URL audit, 4 tasks per site, live agent view, diagnosis, score,
peer comparison (signals only), leaderboard of benchmarked sites, DEMO_MODE.
```

```
OUT (roadmap only): logins, scheduled monitoring, PDF export, user accounts,
auto-generated pull requests, non-English sites.

## 4. Pipeline (app/audit.py)
1. signals   : TinyFish Fetch on /, /robots.txt, /llms.txt, /sitemap.xml
2. classify  : LLM on homepage markdown -> site_type, product query, peer
query
3. run tasks : TinyFish Agent, 4 tasks, max 2 concurrent, 150 s cap each
4. diagnose  : LLM on each non-success task's step trace
5. score     : pure Python (section 7)
6. peers     : TinyFish Search(peer query) -> 3 other domains -> signals only
Every TinyFish and LLM call goes through app/cache.py.

## 5. Site types and task library (app/tasks.py)
site_type in [ecommerce, saas, travel, services]
| type      | task ids                                        |
| ecommerce | find_contact, find_policy, find_price, reach_checkout |
| saas      | find_contact, find_policy, find_pricing, reach_signup |
| travel    | find_contact, find_policy, search_options, list_services |
| services  | find_contact, find_policy, list_services, reach_enquiry |
Each task = {id, label, goal_template, expected_steps}.
Goals follow the 6 goal-writing rules. No login, no personal data,
no payment, no form submission, ever.

## 6. Blocker taxonomy
none, cookie_or_popup, captcha_or_bot_block, login_wall, element_not_found,
unlabeled_control, slow_or_dynamic_content, confusing_navigation, other

## 7. Scoring (app/score.py)
outcome value: success 1.0, partial 0.5, failure 0.0
success     = 55 * mean(outcome value)
efficiency  = 20 * mean(min(1, expected_steps / steps) * outcome value)
readability = fetchable 6 + text_rendered 5 + llms_txt 4 + robots_ok 4
              + sitemap 3 + meta 3                        (max 25)
total = round(success + efficiency + readability)
grade: A >= 85, B >= 70, C >= 55, D >= 40, else F
agent_tax.avg_steps = mean(steps over all tasks)
agent_tax.usd_per_task = avg_steps * 0.016

## 8. Signals (app/signals.py)
fetchable     : homepage not in Fetch errors as bot_blocked / login_required
/ http error
text_rendered : homepage text length > 500 characters
llms_txt      : /llms.txt fetched and non-empty, final_url still ends in
llms.txt
robots_ok     : robots.txt exists and does not block all agents or named AI
```

```
agents
sitemap       : /sitemap.xml fetched
meta          : homepage title and description both present

## 9. API (app/main.py)
POST /api/audit            {url}            -> {audit_id}
GET  /api/audit/{id}/events  SSE stream of events (section 10)
GET  /api/report/{domain}                    -> report JSON
GET  /api/leaderboard                        -> [{domain, total, grade,
usd_per_task}]
GET  /                                       -> web/index.html

## 10. Events (SSE, one JSON object per message)
stage        {stage: signals|classify|tasks|diagnose|score|peers}
signals      {signals}
tasks        {tasks: [{id, label}]}
task_update  {task_id, status, steps, streaming_url}
task_done    {task: TaskResult}
diagnosis    {diagnosis}
score        {score, agent_tax}
peers        {peers}
done         {report}
error        {message, recoverable}

## 11. Non-functional
- Full audit under 4 minutes. Cached audit under 3 seconds.
- One failed task never fails the audit.
- DEMO_MODE=1 runs with no keys and no network from data/cache.
- No secrets in git. .env in .gitignore.

## 12. Safety and ethics
Agents only read public pages and never log in, submit, pay or enter personal
data. Each audit makes at most 4 agent runs per site. Benchmark results are
presented as a snapshot, with the date, not as a judgement of the company.
```

8. agents.md (paste into the repo root)

`agents.md` does two jobs. Its top half instructs any coding agent that opens the repo. Its bottom half is an honest log of everything AI generated, which protects us in the originality screening: we show exactly what the AI did and what humans decided and verified.

Logging rules for the team

1. Every time a coding agent writes or changes code, it appends one row to the log before it reports done.
2. A human fills the "Verified by" cell only after running the gate check. No verification, no merge.
3. Prompts are summarised in one line, not pasted in full.
4. Anything an AI suggested that we **rejected** gets logged too, with the reason. Judges read this as engineering judgement.

```
# agents.md

## Part A: Instructions for coding agents working in this repo

Read design.md first. It is the source of truth for scope, data shapes,
scoring and the API. If a request conflicts with design.md, stop and ask.

Rules:
1. Work only on the current phase named by the human. Do not start the next
phase.
2. Keep code short: functions under 30 lines, files under 150 lines,
   one-line docstring at the top of every file, type hints everywhere.
3. All TinyFish calls go in app/tinyfish.py. All LLM calls go through
   app/llm.py ask_json(). Everything external is wrapped with @cached.
4. Never hard-code keys. Never commit .env. Never print secrets.
5. Never weaken safety: agent goals must forbid login, payment, personal
   data and form submission.
6. Before saying "done": run the phase's gate commands, paste the output,
   and append a row to the AI work log below.
7. If a gate check fails twice, stop and report the failure. Do not loop.
8. Do not add dependencies without listing them and the reason.

Run commands:
- install : pip install -r requirements.txt
- server  : uvicorn app.main:app --reload --port 8000
- tests   : pytest -q
- smoke   : python scripts/smoke_tinyfish.py
- format  : ruff format .

## Part B: AI work log

| # | Time (IST) | Phase | Tool and model | What the AI did | Files | Human
decision | Verified by |
|---|------------|-------|----------------|-----------------|-------|--------
--------|-------------|
| 1 | 15:05 | 0 | Claude, playbook | Wrote design.md, agents.md, task plan |
design.md, agents.md | Accepted; trimmed scope to 4 tasks | A |
```

```
| 2 | 15:20 | 0 | (coding agent) | Generated repo skeleton and smoke test |
app/*, scripts/smoke_tinyfish.py | Accepted | A |
|   |       |       |                |                 |       |
|             |

## Part C: AI suggestions we rejected

| Suggestion | Why we rejected it |
|------------|--------------------|
| Use a database for reports | JSON files are enough for a demo and easier to
inspect |
|            |                    |

## Part D: What humans did (not AI)
- Chose the problem, the angle (behavioural testing) and the Agent Tax
metric.
- Designed the scoring weights and blocker taxonomy.
- Picked and reviewed the benchmark sites and checked every diagnosis shown
in the demo.
- Wrote the pitch, recorded the video, made the deck.
```

9. Agent operating rules: phase gates and double-checks

The coding agent follows one loop per phase: **plan, build, self-check, gate, log, commit.** It may not start phase N+1 until a human has seen phase N's gate pass. This is what stops an AI from building ten half-features.

The phase loop

1. **Plan (2 minutes).** The agent restates the phase goal, lists the files it will touch, and lists the gate checks it will run. A human replies "go" or corrects it.
2. **Build.** Only the files listed. Nothing outside the phase.
3. **Self-check.** The agent runs the phase's gate commands itself and pastes the real output. It also re-reads its diff against the coding standards (section 6) and design.md.
4. **Gate.** A human (role E, or the phase owner) runs the same commands independently and ticks the gate checklist in this playbook.
5. **Log.** One row in `agents.md` Part B.
6. **Commit.** `git commit -m "phase N: <what works now>"` and push. Post `GATE Pn PASSED <hash>` in the team chat.

Double-check rules (the agent must obey all of them)

- **Evidence, not claims.** "Done" must be followed by command output that proves it. "It should work" is a failed gate.

- **Run it twice.** Every gate command runs a second time with `DEMO_MODE=1` to prove the cache replays.
- **Check the negative case.** Each phase includes at least one failure-path check (bad URL, blocked site, failed agent run).
- **Diff review.** Before declaring done, the agent lists every changed file with its line count. Any file over 150 lines must be split first.
- **Two strikes.** If a gate fails twice, the agent stops and reports what it tried. A human decides whether to fix, cut scope, or use the section 17 fallback.
- **No silent scope changes.** Anything not in design.md needs a human "yes" and a design.md edit first.

The master prompt (paste at the start of every phase)

```
You are the coding agent for AgentLighthouse. Read design.md and agents.md
first; they are binding.

CURRENT PHASE: <number and name>
PHASE GOAL: <copy from the playbook>
FILES YOU MAY TOUCH: <copy from the playbook>
GATE CHECKS: <copy the checklist from the playbook>

Process:
1. Reply with a short plan: files, functions, gate commands. Wait for "go".
2. Build only this phase. Keep functions under 30 lines and files under
   150 lines. One-line docstring per file. Type hints everywhere.
3. Run every gate check yourself, then again with DEMO_MODE=1.
   Paste the real output.
4. List changed files with line counts.
5. Append a row to agents.md Part B.
6. Say "PHASE <n> READY FOR HUMAN GATE" and stop. Do not start the next
phase.

If any check fails twice, stop and explain what you tried.
```

Definition of done for the whole project

- [ ] `git clone` plus the 3 README commands give a working demo in `DEMO_MODE=1` with no keys.
- [ ] A live audit of a fresh URL finishes in under 4 minutes and shows the live agent view.
- [ ] Report shows score, grade, Agent Tax, 4 task results, at least one AI diagnosis with a fix, and peers.
- [ ] Leaderboard shows 8 to 10 real sites.
- [ ] `pytest -q` passes.

- [ ] README, design.md and agents.md are complete and accurate.
- [ ] Video, social post and Unstop submission are done.

Phase 0: Setup (15:00 to 15:30, owner A)

Goal: a public repo where one command proves TinyFish Fetch, Search, Agent and the LLM all work, and the results are cached.

Steps

1. Create a public GitHub repo `agentlighthouse`. Add `.gitignore` with `.env`, `__pycache__/`, `.venv/`.
2. Add `design.md` and `agents.md` from sections 7 and 8. **This is commit #1**, so the history shows design came before code.
3. Create the folders from section 6, `requirements.txt` and `.env.example`.
4. Claim TinyFish credits, create a key at agent.tinyfish.ai/api-keys, and put it in `.env`. Add the LLM key, base URL and model.
5. Paste `app/tinyfish.py`, `app/cache.py` and `app/llm.py` from sections 5 and 6. Create `app/tasks.py` with just `BLOCKERS` and `SCHEMA` for now.
6. Write and run the smoke test below.

`.env.example`

```
TINYFISH_API_KEY=
LLM_BASE_URL=https://api.openai.com/v1   # or any OpenAI-compatible endpoint
LLM_API_KEY=
LLM_MODEL=
DEMO_MODE=0
```

`scripts/smoke_tinyfish.py`

```
"""Phase 0 check: one Fetch, one Search, one tiny agent run, one LLM call."""
import asyncio
from dotenv import load_dotenv

load_dotenv()  # before importing app modules: they read keys at import
from app import llm, tinyfish  # noqa: E402
from app.tasks import SCHEMA  # noqa: E402

GOAL = ("Report the main heading of this page in 'answer'. "
        "Set outcome to success if you found it, and blocker to none.")


async def main() -> None:
```

```
    f = await tinyfish.fetch(["https://example.com"])
    print("FETCH OK :", f["results"][0]["title"])
    s = await tinyfish.search("online bookstore india")
    print("SEARCH OK:", [r["url"] for r in s[:3]])
    run = await tinyfish.run_agent("https://example.com", GOAL, SCHEMA)
    print("AGENT    :", run["status"], run.get("num_of_steps"),
tinyfish.parse_result(run))
    print("LLM OK   :", await llm.ask_json("Reply only in JSON.", 'Return
{"ok": true}'))


asyncio.run(main())
```

Run it with `python -m scripts.smoke_tinyfish` from the repo root (add an empty `scripts/__init__.py` and `app/__init__.py`).

Parallel work while A sets up

- **B** drafts the goal text for all 10 tasks in a scratch file (Phase 1 uses it).
- **C** reads section 10 and starts the UI with fake data from the data contract in section 4.
- **D** creates the README skeleton (section 11) and the deck outline (section 14).
- **E** writes `data/sites.txt` with 10 public sites: 2 big marketplaces, 3 D2C brands, 2 SaaS products, 2 travel sites and 1 public-service portal.

Gate P0 (all must pass before Phase 1)

- [ ] Repo is public; the first commit contains only design.md, agents.md and .gitignore.
- [ ] Smoke test prints 4 lines: FETCH OK, SEARCH OK, AGENT COMPLETED with a parsed result, LLM OK.
- [ ] `data/cache/` now holds 4 JSON files.
- [ ] `DEMO_MODE=1 python -m scripts.smoke_tinyfish` prints the same 4 lines with Wi-Fi off.
- [ ] `git grep TINYFISH_API_KEY=` shows only `.env.example` with an empty value.
- [ ] agents.md log has rows 1 and 2. Commit: `phase 0: skeleton, TinyFish and LLM smoke test pass`.

Phase 1: Task library and agent runner (15:30 to 16:45, owners A and B)

Goal: given a URL and a task id, a TinyFish agent attempts the task and we get back one clean `TaskResult`: outcome, blocker, answer, steps, seconds, run id and the last 15 step actions.

Files: `app/tasks.py`, `app/runner.py` (new, ~45 lines), `scripts/try_task.py`.

`app/tasks.py`

```
"""Task library: which jobs an AI agent should try on each kind of site."""
BLOCKERS = [...]  # from section 5
SCHEMA = {...}    # from section 5

SAFETY = ("Browse anonymously. Never log in, create an account, enter
personal or "
          "payment details, or submit any form. If a cookie banner or popup
appears, "
          "try to close it and continue.")
REPORT = ("Set outcome to 'success' if you {done}, 'partial' if you got close
but could "
          "not finish, otherwise 'failure'. Set blocker to the single main
obstacle you "
          "hit, or 'none', and describe it in blocker_detail. Put what you
found in answer.")

TASKS = {
    "find_contact": {"label": "Find support contact", "steps": 6, "done":
"found a contact method",
                     "ask": "Find how to contact customer support: an email,
phone number or contact page."},
    "find_policy": {"label": "Find refund policy", "steps": 6, "done": "found
the policy and its time window",
                    "ask": "Find the return, refund or cancellation policy
and state the time window it allows."},
    "find_price": {"label": "Find a product price", "steps": 8, "done":
"found a product name and price",
                   "ask": "Use the site's search or menus to find a product
matching '{query}' and report its name and price."},
    "reach_checkout": {"label": "Reach checkout", "steps": 14, "done":
"reached checkout or sign-in-to-checkout",
                       "ask": "Add any in-stock product to the cart and go to
checkout. Stop as soon as the checkout or sign-in-to-checkout page is
visible."},
    "find_pricing": {"label": "Find plan pricing", "steps": 6, "done":
"listed plan names and prices",
                     "ask": "Find the pricing page and list each plan name
with its monthly price."},
    "reach_signup": {"label": "Reach sign-up", "steps": 6, "done": "reached
the sign-up form",
                     "ask": "Find the sign-up or free-trial page. Stop when
the sign-up form is visible."},
    "search_options": {"label": "Search options", "steps": 15, "done":
```

```
"reported the cheapest option and price",
                       "ask": "Search for '{query}' and report the cheapest
option shown with its price."},
    "list_services": {"label": "List offerings", "steps": 5, "done": "listed
three offerings",
                      "ask": "Find what this business offers and list three
of its products or services."},
    "reach_enquiry": {"label": "Reach enquiry form", "steps": 8, "done":
"reached the enquiry or booking form",
                      "ask": "Find the page to book, enquire or request a
quote. Stop when the form is visible."},
}

SITE_TASKS = {
    "ecommerce": ["find_contact", "find_policy", "find_price",
"reach_checkout"],
    "saas": ["find_contact", "find_policy", "find_pricing", "reach_signup"],
    "travel": ["find_contact", "find_policy", "search_options",
"list_services"],
    "services": ["find_contact", "find_policy", "list_services",
"reach_enquiry"],
}


def goal(task_id: str, query: str) -> str:
    t = TASKS[task_id]
    return ("You are testing whether an AI agent can use this website. "
            f"Task: {t['ask'].format(query=query)} {SAFETY}
{REPORT.format(done=t['done'])}")
```

app/runner.py

```
"""Run one task with a TinyFish agent and turn the run into a TaskResult."""
import asyncio
from datetime import datetime
from app import tasks, tinyfish

SLOTS = asyncio.Semaphore(2)  # pay-as-you-go allows 2 agents at once


def _seconds(run: dict) -> float:
    t = [run.get(k) for k in ("started_at", "finished_at")]
    if not all(t):
        return 0.0
    a, b = (datetime.fromisoformat(x.replace("Z", "+00:00")) for x in t)
    return round((b - a).total_seconds(), 1)
```

```
async def run_task(url: str, task_id: str, query: str, emit) -> dict:
    async def on_update(run: dict) -> None:
        await emit("task_update", task_id=task_id, status=run["status"],
                   steps=len(run.get("steps") or []),
streaming_url=run.get("streaming_url"))

    try:
        async with SLOTS:
            run = await tinyfish.run_agent(url, tasks.goal(task_id, query),
tasks.SCHEMA,
                                           on_update=on_update)
    except Exception as e:  # one bad task must never crash the audit
        run = {"status": "FAILED", "error": {"message": str(e)}}
    steps = run.get("steps") or []
    return {"id": task_id, "label": tasks.TASKS[task_id]["label"],
            "expected_steps": tasks.TASKS[task_id]["steps"],
**tinyfish.parse_result(run),
            "steps": run.get("num_of_steps") or len(steps), "seconds":
_seconds(run),
            "run_id": run.get("run_id"), "trace": [s.get("action") for s in
steps][-15:]}
```

`scripts/try_task.py` runs one task from the command line and prints the result:

```
"""Usage: python -m scripts.try_task <url> <task_id> [query]"""
import asyncio, json, sys
from dotenv import load_dotenv

load_dotenv()
from app.runner import run_task  # noqa: E402


async def show(event: str, **data) -> None:
    print(event, data.get("status"), data.get("steps"))


url, task_id, query = sys.argv[1], sys.argv[2], (sys.argv[3:] or
["bestseller"])[0]
print(json.dumps(asyncio.run(run_task(url, task_id, query, show)), indent=2))
```

Prompting tips for this phase

- If a task keeps returning `failure` on a site where a human can do it in 20 seconds, the goal is probably vague. Tighten `ask` before blaming the site.

- Watch one run's `streaming_url` in a browser tab. You will learn more about how the agent behaves in 2 minutes than from logs.
- Test two kinds of site: one simple (a small D2C store) and one heavy (a large marketplace).

Gate P1

- [ ] `python -m scripts.try_task <shop url> find_price "running shoes"` prints a TaskResult with `outcome`, `blocker`, `steps` > 0 and a non-empty `trace`.
- [ ] Running the same command again returns in under 1 second (cache hit) and prints the identical result.
- [ ] A second task (`find_policy`) works on the same site.
- [ ] Negative case: `python -m scripts.try_task https://this-domain-does-not-exist-xyz.com find_contact` returns `outcome: failure` and does not crash.
- [ ] While a run is going, the printed updates show the status changing from PENDING or RUNNING to COMPLETED.
- [ ] Every goal string contains the SAFETY sentence (check `tasks.goal` output by eye).
- [ ] Commit: `phase 1: agent task runner with cache and parsed results`.

Phase 2: Machine-readability signals, task planning and peers (16:45 to 17:30, owner B)

Goal: for any URL, produce three things using TinyFish Fetch, the LLM and TinyFish Search: the six readability signals, a task plan (site type, search query, task ids), and 3 competitor domains with their signals.

Files: `app/signals.py` , `app/score.py` (only `readability()` for now), `tasks.py` (add `plan()`), `tests/test_signals.py` .

`app/signals.py`

```
"""Machine-readability signals and competitor discovery via TinyFish Fetch
and Search."""
import asyncio, re
from urllib.parse import urlparse
from app import tinyfish
from app.score import readability

PATHS = ["/", "/robots.txt", "/llms.txt", "/sitemap.xml"]
BAD = {"bot_blocked", "login_required", "target_http_error",
"target_unreachable", "timeout"}
AI_BOTS = ("\\*", "gptbot", "claudebot", "perplexitybot", "google-extended")
SKIP = ("wikipedia", "youtube", "reddit", "quora", "facebook", "instagram",
```

```
"linkedin", "medium")


def root(url: str) -> str:
    p = urlparse(url if "://" in url else f"https://{url}")
    return f"{p.scheme}://{p.netloc}"


def robots_ok(text: str) -> bool:
    """robots.txt exists and does not disallow everything for all or for AI
agents."""
    t = text.lower()
    return bool(t) and not any(
        re.search(rf"user-agent:\s*{bot}\s*\ndisallow:\s*/\s*$", t, re.M) for
bot in AI_BOTS)


def signals_from(base: str, out: dict) -> dict:
    """Pure function: Fetch output -> signals. Unit-tested."""
    got = {r["url"]: r for r in out.get("results", [])}
    err = {e["url"]: e["error"] for e in out.get("errors", [])}
    home, llms = got.get(base + "/", {}), got.get(base + "/llms.txt", {})
    return {
        "fetchable": err.get(base + "/") not in BAD,
        "text_rendered": len(home.get("text") or "") > 500,
        "llms_txt": len(llms.get("text") or "") > 20
                    and (llms.get("final_url") or "").endswith("llms.txt"),
        "robots_ok": robots_ok((got.get(base + "/robots.txt") or
{}).get("text") or ""),
        "sitemap": base + "/sitemap.xml" in got,
        "meta": bool(home.get("title") and home.get("description")),
        "fetch_error": err.get(base + "/"),
    }


async def read_site(url: str) -> tuple[dict, str]:
    base = root(url)
    out = await tinyfish.fetch([base + p for p in PATHS])
    home = next((r for r in out["results"] if r["url"] == base + "/"), {})
    return signals_from(base, out), (home.get("text") or "")[:4000]


async def peers(url: str, query: str, n: int = 3) -> list[dict]:
    mine = urlparse(root(url)).netloc.removeprefix("www.")
    found = [urlparse(r["url"]).netloc.removeprefix("www.") for r in await
tinyfish.search(query)]
```

```
    picks = list(dict.fromkeys(d for d in found if d and d != mine
                               and not any(s in d for s in SKIP)))[:n]
    sigs = await asyncio.gather(*(read_site(f"https://{d}") for d in picks))
    return [{"domain": d, "signals": s, "readability": readability(s)}
            for d, (s, _) in zip(picks, sigs)]
```

Add to `app/score.py`

```
WEIGHTS = {"fetchable": 6, "text_rendered": 5, "llms_txt": 4, "robots_ok": 4,
"sitemap": 3, "meta": 3}


def readability(signals: dict) -> int:
    return sum(w for k, w in WEIGHTS.items() if signals.get(k))
```

Add to `app/tasks.py`

```
from app import llm

CLASSIFY = ("You classify websites for an AI-agent usability audit. Reply in
JSON with: "
            "site_type (one of ecommerce, saas, travel, services); query (a
short, realistic "
            "search a customer would type on THIS site); peer_query (a web
search that finds "
            "this site's direct competitors in the same country); summary
(one sentence).")


async def plan(url: str, homepage_md: str) -> dict:
    p = await llm.ask_json(CLASSIFY, f"URL:
{url}\n\nHomepage:\n{homepage_md}")
    if p.get("site_type") not in SITE_TASKS:
        p["site_type"] = "services"
    return {**p, "task_ids": SITE_TASKS[p["site_type"]]}
```

`tests/test_signals.py` (no network, runs in 1 second)

```
from app.signals import robots_ok, root, signals_from


def test_root():
    assert root("shop.in/x?y=1") == "https://shop.in"


def test_robots():
    assert robots_ok("User-agent: *\nDisallow: /admin")
    assert not robots_ok("User-agent: *\nDisallow: /")
    assert not robots_ok("User-agent: GPTBot\nDisallow: /")
    assert not robots_ok("")


def test_bot_blocked_home():
    s = signals_from("https://a.com", {"results": [], "errors": [{"url":
"https://a.com/", "error": "bot_blocked"}]})
    assert not s["fetchable"] and s["fetch_error"] == "bot_blocked"
```

Gate P2

- [ ] `pytest -q tests/test_signals.py` passes.
- [ ] `read_site()` on 3 benchmark sites prints six true/false signals each; values look right when checked by hand in a browser (open `/robots.txt` and `/llms.txt` yourself).
- [ ] `plan()` gives a sensible `site_type` and `query` for one shop, one SaaS and one travel site.
- [ ] `peers()` returns 3 domains, none equal to the audited domain and none from the SKIP list.
- [ ] Second run of all of the above is instant (cache).
- [ ] Commit: `phase 2: signals, task planning, competitor discovery`.

Phase 3: Score, AI diagnosis and the full pipeline (17:30 to 18:45, owners B and A)

Goal: `python -m scripts.audit_cli <url>` runs all six steps and saves a complete report JSON to `data/reports/<domain>.json`. This is the heart of the product; after this gate, everything else is presentation.

Files: `app/score.py`, `app/diagnose.py`, `app/audit.py`, `scripts/audit_cli.py`, `scripts/benchmark.py`, `tests/test_score.py`.

`app/score.py` (complete)

```
"""Agent-Readiness Score, grade and Agent Tax. Pure functions, unit-
tested."""
from statistics import mean

OUTCOME = {"success": 1.0, "partial": 0.5, "failure": 0.0}
WEIGHTS = {"fetchable": 6, "text_rendered": 5, "llms_txt": 4, "robots_ok": 4,
"sitemap": 3, "meta": 3}
USD_PER_STEP = 0.016  # TinyFish published per-step price


def readability(signals: dict) -> int:
    return sum(w for k, w in WEIGHTS.items() if signals.get(k))


def grade(total: int) -> str:
    return next(g for g, cut in (("A", 85), ("B", 70), ("C", 55), ("D", 40),
("F", 0)) if total >= cut)


def score(tasks: list[dict], signals: dict) -> dict:
    val = [OUTCOME.get(t.get("outcome"), 0.0) for t in tasks] or [0.0]
    eff = [v * min(1, t["expected_steps"] / max(t["steps"], 1)) for v, t in
zip(val, tasks)] or [0.0]
    s, e, r = 55 * mean(val), 20 * mean(eff), readability(signals)
    total = round(s + e + r)
    return {"total": total, "grade": grade(total), "success": round(s),
            "efficiency": round(e), "readability": r}


def agent_tax(tasks: list[dict]) -> dict:
    steps = mean([t["steps"] for t in tasks]) if tasks else 0
    return {"avg_steps": round(steps, 1), "usd_per_task": round(steps *
USD_PER_STEP, 3)}
```

`app/diagnose.py`, the core AI/ML step

```
"""LLM reads a failed or slow agent run and explains the blocker and the
fix."""
import asyncio, json
from app import llm
from app.tasks import BLOCKERS

SYSTEM = f"""You are a senior web engineer explaining why an AI web agent
struggled on a website.
You get the task, the agent's own report and its last browser actions. Reply
in JSON with:
```

```
blocker (one of {BLOCKERS}); root_cause (1-2 specific sentences; name the UI
element if the
trace mentions it); fix (a concrete change for the site's developers, with a
short HTML, ARIA
or robots.txt snippet when useful); severity (high if it blocks purchase or
sign-up, medium if
it slows the agent a lot, else low); confidence (0 to 1).
Use only evidence in the input. If the trace is too thin, write "insufficient
evidence" as
root_cause and set confidence below 0.4."""

FIELDS = ("label", "outcome", "blocker", "blocker_detail", "answer", "steps",
          "expected_steps", "final_url", "trace")


def needs_diagnosis(t: dict) -> bool:
    return t.get("outcome") != "success" or t["steps"] > 2 *
t["expected_steps"]


async def diagnose(task: dict, url: str) -> dict:
    evidence = {k: task.get(k) for k in FIELDS} | {"site": url}
    d = await llm.ask_json(SYSTEM, json.dumps(evidence))
    if d.get("blocker") not in BLOCKERS:
        d["blocker"] = task.get("blocker") or "other"
    return {"task_id": task["id"], **d}


async def diagnose_all(tasks: list[dict], url: str) -> list[dict]:
    return await asyncio.gather(*(diagnose(t, url) for t in tasks if
needs_diagnosis(t)))
```

`app/audit.py`, the pipeline

```
"""The 6-step audit pipeline. Emits events as it goes; returns and saves the
report."""
import asyncio, json, pathlib
from datetime import datetime, timezone
from urllib.parse import urlparse
from app import diagnose, score, signals, tasks
from app.runner import run_task

REPORTS = pathlib.Path("data/reports")
REPORTS.mkdir(parents=True, exist_ok=True)
```

```
async def _task(url: str, task_id: str, query: str, emit) -> dict:
    result = await run_task(url, task_id, query, emit)
    await emit("task_done", task=result)
    return result


async def _peers(url: str, query: str) -> list[dict]:
    try:
        return await signals.peers(url, query)
    except Exception:
        return []  # peers are a bonus, never a blocker


async def audit(url: str, emit) -> dict:
    await emit("stage", stage="signals")
    sig, home = await signals.read_site(url)
    await emit("signals", signals=sig)
    await emit("stage", stage="classify")
    plan = await tasks.plan(url, home)
    await emit("tasks", tasks=[{"id": t, "label": tasks.TASKS[t]["label"]}
for t in plan["task_ids"]])
    await emit("stage", stage="tasks")
    peers = asyncio.create_task(_peers(url, plan.get("peer_query", "")))
    results = await asyncio.gather(*(_task(url, t, plan.get("query", ""),
emit) for t in plan["task_ids"]))
    await emit("stage", stage="diagnose")
    diags = await diagnose.diagnose_all(results, url)
    for d in diags:
        await emit("diagnosis", diagnosis=d)
    report = {"url": url, "domain": urlparse(signals.root(url)).netloc,
              "audited_at":
datetime.now(timezone.utc).isoformat(timespec="seconds"),
              "site_type": plan["site_type"], "summary": plan.get("summary"),
"signals": sig,
              "tasks": results, "diagnoses": diags, "score":
score.score(results, sig),
              "agent_tax": score.agent_tax(results), "peers": await peers}
    await emit("score", score=report["score"], agent_tax=report["agent_tax"])
    await emit("peers", peers=report["peers"])
    (REPORTS / f"{report['domain']}.json").write_text(json.dumps(report,
indent=2))
    await emit("done", report=report)
    return report
```

`scripts/audit_cli.py` and **`scripts/benchmark.py`**

```
"""Usage: python -m scripts.audit_cli <url>"""
import asyncio, sys
from dotenv import load_dotenv

load_dotenv()
from app.audit import audit  # noqa: E402


async def show(event: str, **d) -> None:
    print(f"{event:12}", d.get("stage") or d.get("task_id") or d.get("score")
or "")


r = asyncio.run(audit(sys.argv[1], show))
print("\nSCORE", r["score"]["total"], r["score"]["grade"], "| Agent Tax",
r["agent_tax"])
```

```
"""Audit every URL in data/sites.txt, one at a time, and write
data/leaderboard.json."""
import asyncio, json, pathlib
from dotenv import load_dotenv

load_dotenv()
from app.audit import audit  # noqa: E402


async def quiet(event: str, **d) -> None:
    if event in ("stage", "task_done"):
        print(event, d.get("stage") or d["task"]["id"], d.get("task",
{}).get("outcome", ""))


async def main() -> None:
    rows = []
    for url in pathlib.Path("data/sites.txt").read_text().split():
        try:
            r = await audit(url, quiet)
            rows.append({"domain": r["domain"], "site_type": r["site_type"],
**r["score"], **r["agent_tax"]})
        except Exception as e:
            print("SKIPPED", url, e)
    rows.sort(key=lambda r: -r["total"])
    pathlib.Path("data/leaderboard.json").write_text(json.dumps(rows,
```

```
indent=2))


asyncio.run(main())
```

`tests/test_score.py`

```
from app.score import agent_tax, grade, score

ALL = {k: True for k in ("fetchable", "text_rendered", "llms_txt",
"robots_ok", "sitemap", "meta")}


def task(outcome, steps=6, expected=6):
    return {"outcome": outcome, "steps": steps, "expected_steps": expected}


def test_perfect_site_scores_100():
    assert score([task("success")] * 4, ALL)["total"] == 100


def test_dead_site_scores_0():
    assert score([task("failure", 0)] * 4, {})["total"] == 0


def test_slow_success_loses_efficiency():
    assert score([task("success", 24)] * 4, ALL)["efficiency"] == 5


def test_grades_and_tax():
    assert [grade(x) for x in (85, 70, 55, 40, 39)] == ["A", "B", "C", "D",
"F"]
    assert agent_tax([task("success", 10)])["usd_per_task"] == 0.16
```

Optional, high-value for Technique points: a 10-case diagnosis check. Role E reads 10 failed runs (watch the recording or read the trace) and writes the true blocker by hand. Compare with the LLM's `blocker`. Report the agreement in the README, for example "LLM blocker label matched human review in 8 of 10 cases." Judges rarely see a team measure its own AI.

Gate P3

- [ ] `pytest -q` passes (score and signals tests).
- [ ] `python -m scripts.audit_cli <shop url>` finishes in under 4 minutes and writes `data/reports/<domain>.json` with every key from the data contract.

- [ ] At least one task is diagnosed, and the diagnosis names a specific UI element or page, not generic advice. If it's generic, tighten the SYSTEM prompt.
- [ ] Read the score by hand: does a site that failed checkout score lower than one that succeeded? If not, fix the weights now, not after the demo.
- [ ] Re-run with `DEMO_MODE=1`: identical report, under 3 seconds.
- [ ] Commit: `phase 3: full audit pipeline, score, AI diagnosis`.

Phase 4: API with live streaming (18:45 to 19:30, owner A)

Goal: the browser can start an audit and receive every pipeline event live over Server-Sent Events, including each agent's `streaming_url`. Reports and the leaderboard are served as JSON, and the UI is served from `/`.

File: `app/main.py` (complete)

```
"""HTTP API: start an audit, stream its events, serve reports and the UI."""
import asyncio, json, pathlib, uuid
from dotenv import load_dotenv

load_dotenv()
from fastapi import FastAPI, HTTPException  # noqa: E402
from fastapi.responses import StreamingResponse  # noqa: E402
from fastapi.staticfiles import StaticFiles  # noqa: E402
from pydantic import BaseModel, field_validator  # noqa: E402
from app.audit import audit  # noqa: E402

app = FastAPI(title="AgentLighthouse")
DATA = pathlib.Path("data")
QUEUES: dict[str, asyncio.Queue] = {}
JOBS: set[asyncio.Task] = set()  # keep references so jobs are not garbage-
collected


class AuditIn(BaseModel):
    url: str

    @field_validator("url")
    @classmethod
    def http_only(cls, v: str) -> str:
        v = v.strip() if "://" in v else f"https://{v.strip()}"
        if not v.startswith(("http://", "https://")):
            raise ValueError("url must be http or https")
        return v
```

```
@app.post("/api/audit")
async def start(body: AuditIn) -> dict:
    audit_id = uuid.uuid4().hex[:8]
    queue = QUEUES[audit_id] = asyncio.Queue()

    async def emit(event: str, **data) -> None:
        await queue.put({"type": event, **data})

    async def job() -> None:
        try:
            await audit(body.url, emit)
        except Exception as e:
            await emit("error", message=str(e), recoverable=False)
        finally:
            await queue.put(None)  # end of stream

    task = asyncio.create_task(job())
    JOBS.add(task)
    task.add_done_callback(JOBS.discard)
    return {"audit_id": audit_id}


@app.get("/api/audit/{audit_id}/events")
async def events(audit_id: str) -> StreamingResponse:
    queue = QUEUES.get(audit_id)
    if queue is None:
        raise HTTPException(404, "unknown audit")

    async def stream():
        while (msg := await queue.get()) is not None:
            yield f"data: {json.dumps(msg)}\n\n"
        QUEUES.pop(audit_id, None)

    return StreamingResponse(stream(), media_type="text/event-stream")


def _json(path: pathlib.Path, default):
    return json.loads(path.read_text()) if path.exists() else default


@app.get("/api/report/{domain}")
def report(domain: str) -> dict:
    data = _json(DATA / "reports" / f"{pathlib.Path(domain).name}.json",
None)
    if data is None:
```

```
        raise HTTPException(404, "no report for this domain")
    return data


@app.get("/api/leaderboard")
def leaderboard() -> list:
    return _json(DATA / "leaderboard.json", [])


app.mount("/", StaticFiles(directory="web", html=True), name="web")
```

`pathlib.Path(domain).name` stops path tricks like `../../.env` in the report route.

Gate P4

- [ ] `uvicorn app.main:app --port 8000` starts with no errors.
- [ ] `curl -s -X POST localhost:8000/api/audit -H 'content-type: application/json' -d '{"url":"<an already-audited site>"}'` returns an `audit_id`.
- [ ] `curl -N localhost:8000/api/audit/<id>/events` streams `stage`, `signals`, `tasks`, `task_done`, `diagnosis`, `score`, `peers`, `done`, then closes. Cached site: under 3 seconds.
- [ ] A fresh site shows `task_update` events containing a `streaming_url`. Open that URL in a browser tab and you see the agent's live browser.
- [ ] Negative cases: unknown audit id gives 404; `{"url": "ftp://x"}` gives 422; `/api/report/..%2F..%2F.env` gives 404.
- [ ] `curl localhost:8000/api/leaderboard` returns `[]` or the leaderboard.
- [ ] Commit: `phase 4: API with live SSE events`.

Phase 5: UI (19:30 to 21:00, owner C)

Goal: one page that tells the story in the order the demo video needs: paste URL, watch agents work live, see the score appear, read the diagnoses, compare with peers, open the leaderboard. C started this at 15:30 with fake data, so this phase is wiring to the real SSE stream and polishing.

Files: `web/index.html`, `web/app.js`. Give the coding agent the UI prompt in section 10 together with the master prompt.

The SSE wiring (the only tricky part)

```
// web/app.js: start an audit and route each server event to a render
function
async function startAudit(url) {
  const res = await fetch("/api/audit", {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({ url }),
  });
  if (!res.ok) return showError("That doesn't look like a valid website
address.");
  const { audit_id } = await res.json();
  const stream = new EventSource(`/api/audit/${audit_id}/events`);
  stream.onmessage = (msg) => {
    const e = JSON.parse(msg.data);
    (render[e.type] || (() => {}))(e);
    if (e.type === "done" || e.type === "error") stream.close();
  };
  stream.onerror = () => stream.close(); // server closed the stream: audit
finished
}

const render = {
  stage: (e) => setStage(e.stage),
  signals: (e) => drawSignals(e.signals),
  tasks: (e) => drawTaskCards(e.tasks),
  task_update: (e) => updateTaskCard(e), // shows the live iframe when
streaming_url arrives
  task_done: (e) => finishTaskCard(e.task),
  diagnosis: (e) => addDiagnosis(e.diagnosis),
  score: (e) => revealScore(e.score, e.agent_tax),
  peers: (e) => drawPeers(e.peers),
  done: (e) => showShareBar(e.report),
  error: (e) => showError(e.message),
};
```

Demo-proofing rules for the UI

1. **Two demo buttons under the URL box,** "Try a sample shop" and "Try a sample SaaS," which audit sites already in the cache. The demo can never hang on stage.
2. **The live iframe must have a fallback.** If no `streaming_url` arrives within 10 seconds, show the step counter and the last action text instead of an empty box.
3. **Never show raw JSON or stack traces.** Every error becomes one friendly sentence and a "Try again" button.
4. **Animate the score count-up** (0 to the final number in about 1.2 seconds). This is the moment the video cuts to.

Gate P5

- [ ] Cached sample: the whole report renders in under 3 seconds with no console errors.
- [ ] Fresh URL: at least one live agent iframe is visible while tasks run, and task cards change from "Running" to their final state.
- [ ] Every diagnosis card shows blocker, root cause, fix and severity, and the fix snippet is in a copyable code box.
- [ ] Leaderboard view loads from `/api/leaderboard` and is sorted by score.
- [ ] At 390 px width (phone), nothing overflows sideways and the score card is readable.
- [ ] Error state checked: enter `not a url` and see the friendly message.
- [ ] Take 4 screenshots now (input, live agents, report, leaderboard) for README, deck and post.
- [ ] Commit: `phase 5: UI with live agent view, report, leaderboard`.

Phase 6: Freeze, benchmark and ship (21:00 to 23:00, whole team)

Goal: everything the jury opens works, is public, and tells the same story. **Feature freeze at 21:00:** after that, only bug fixes, docs and media.

21:00 to 21:30: Benchmark (B and E)

1. Run `python -m scripts.benchmark` on the 9 to 10 sites in `data/sites.txt`. Run it on one laptop only, so credits are not double-spent.
2. Spot-check 3 reports by hand: does each diagnosis match what the recording shows?
3. Commit `data/cache/`, `data/reports/` and `data/leaderboard.json`. This is what makes `DEMO_MODE=1` work for judges.
4. Pull out 3 headline numbers for the post and deck, for example: "X of 10 sites stopped an agent before checkout", "the most common blocker was Y", "Agent Tax ranged from $A to $B per task." **Use only numbers from our own run.**

21:00 to 22:00: Media (C and D, in parallel)

1. Record the demo video with the script in section 12. Record from a cached sample plus one fresh live run (credits allowing).
2. Make a 15-second GIF of the live agent view plus score reveal for the README top and the post.
3. Fill the deck from section 14.

21:30 to 22:15: README and repo polish (A and D)

1. Fill the README from section 11, including the TinyFish table and the benchmark results.

2. Check `agents.md` is complete and the AI work log has a row per phase.
3. Test a clean clone in a fresh folder: `git clone`, `pip install -r requirements.txt`, `DEMO_MODE=1 uvicorn app.main:app`, open the page, click a sample. **This is the single most important check of the night.**

22:15 to 23:00: Publish and submit (D)

1. Upload the video (YouTube unlisted or Google Drive, "anyone with the link"). Open the link in a private window to confirm it plays.
2. Publish the LinkedIn post and X thread (section 13). Open them logged-out to confirm they are public.
3. Submit on Unstop: repo link, video link, post link. Take a screenshot of the confirmation.
4. Post `SUBMITTED` in the team chat with the screenshot.

Gate P6 (the final gate)

- [ ] Clean clone plus `DEMO_MODE=1` works on a second laptop.
- [ ] Repo, video and post links all open in a private browser window.
- [ ] README contains: GIF, what and why, quick start, how TinyFish is used, architecture diagram, benchmark results, limitations, roadmap, team.
- [ ] Unstop submission confirmation screenshot saved.
- [ ] Final commit: `phase 6: benchmark results, docs, submission`.

10. The UI prompt (copy-paste ready)

Paste this whole block into the coding agent (or v0, Lovable, Cursor) after the master prompt. It is written so the output needs almost no back-and-forth: it fixes the story, layout, visual language, states, data shapes and the exact function names `app.js` must expose.

```
Build the complete front end for AgentLighthouse as two files: web/index.html
and
web/app.js. No build step. Tailwind via CDN, one Google font (Inter), vanilla
JS only.

## What the product is
AgentLighthouse tests whether AI agents can actually use a website. The user
pastes
a URL; real TinyFish web agents try 4 tasks on it (e.g. "find a product
price",
"reach checkout"); we show them working live, then a 0-100 Agent-Readiness
Score,
an "Agent Tax" (cost per task for an AI agent), AI diagnoses of each failure
```

```
with
a copy-paste fix, and a comparison with competitors.
Audience: e-commerce owners, developers, and hackathon judges watching a demo
video.
The page must make sense to someone who sees it for 10 seconds with no sound.

## Visual direction
- Metaphor: a lighthouse beam sweeping a dark sea. Calm, precise, confident.
- Dark theme by default: background #0B1020, surfaces #121933, borders
#1F2A4D,
  text #E6EAF5, muted text #8A94B8. One accent: beam amber #FFB547 for
primary
  actions and the score. Status colours: success #3DDC97, partial #FFB547,
  failure #FF6B6B. Never use colour alone: every status also has an icon and
a word.
- Typography: Inter. Score number 72px semibold with tabular numerals.
Section
  titles 20px semibold. Body 15px. Code in a monospace stack.
- Generous spacing (8px grid), 16px radius cards, 1px borders, no heavy
shadows,
  no gradients except one soft amber radial glow behind the hero score.
- Motion: subtle and meaningful only. Score counts up over 1.2 s. Task cards
pulse
  softly while running. Respect prefers-reduced-motion (disable all
animation).

## Layout (single page, top to bottom)
1. Header: wordmark "AgentLighthouse" with a small lighthouse icon (inline
SVG,
   simple geometric), nav links "Audit" and "Leaderboard", and a small badge
   "Powered by TinyFish web agents".
2. Hero: headline "Can AI agents actually use your website?" Sub-line: "We
send
   real web agents to shop, search and sign up on your site, then show you
exactly
   where they get stuck." A large URL input with placeholder "yourstore.com"
and
   an amber button "Run audit". Under it, two ghost buttons: "Try a sample
shop"
   and "Try a sample SaaS" (they call startAudit with URLs from SAMPLE_SHOP
and
   SAMPLE_SAAS constants at the top of app.js).
3. Progress rail (appears after start): 6 steps in a horizontal stepper:
   Read site, Plan tasks, Run agents, Diagnose, Score, Compare. Current step
   highlighted amber; done steps get a check. Driven by setStage(stage) where
   stage is one of signals|classify|tasks|diagnose|score|peers.
```

```
4. Live agents (the hero moment): a 2x2 grid of task cards. Each card: task
label,
   status chip (Queued, Running, Success, Partial, Failed), step counter
   ("Step 7"), and a 16:9 live view area. When a task_update arrives with a
   streaming_url, load it in an iframe in that card (sandbox="allow-scripts
   allow-same-origin", loading="lazy", title="Live agent browser: <task
label>").
   If no streaming_url within 10 s, show the step counter large and the text
   "Agent is working..." instead of an empty box. When the task finishes,
replace
   the iframe with a result summary: outcome, steps vs expected, seconds, the
   agent's answer (if any), and the blocker as a chip.
5. Score card: big score with grade letter (A-F) in a ring that fills to the
score.
   Three sub-bars: Task success (out of 55), Efficiency (out of 20),
   Machine readability (out of 25). Next to it, the Agent Tax tile: "avg
steps per
   task" and "$ per task for an AI agent", with a one-line explainer: "What
every
   AI agent pays, in steps, to get things done on your site."
6. Signals strip: 6 small pills with check or cross: Reachable by agents,
Content
   renders without JS, llms.txt, AI-friendly robots.txt, Sitemap, Title and
   description.
7. Diagnoses: a list of cards, highest severity first. Each card: severity
chip,
   task label, blocker name in plain English (map ids to words, e.g.
   cookie_or_popup -> "Cookie banner or popup"), root cause paragraph, "Fix"
block
   in a code box with a Copy button (shows "Copied" for 1.5 s), and
confidence as
   a small muted percentage.
8. Peers: a compact comparison table: domain, machine-readability /25, and a
   horizontal bar; the audited site highlighted amber in the same table.
9. Share bar (after done): "Copy report link" (uses /api/report/<domain>) and
   "Download JSON".
10. Leaderboard view (nav link, same page, toggled section): table from
    GET /api/leaderboard with rank, domain, site type, score, grade chip,
    avg steps, $ per task. Sorted by score. A muted note: "Snapshot of public
    pages on <date>. Scores measure AI-agent usability, not business
quality."
11. Footer: "Built with TinyFish Agent, Fetch and Search" and a GitHub link.

## Data and events (do not change these names)
app.js must expose: startAudit(url), setStage(stage), drawSignals(signals),
drawTaskCards(tasks), updateTaskCard(e), finishTaskCard(task),
```

```
addDiagnosis(d),
revealScore(score, agentTax), drawPeers(peers), showShareBar(report),
showError(message), loadLeaderboard().
Events arrive from EventSource('/api/audit/<id>/events') as JSON with a
"type"
field: stage, signals, tasks, task_update, task_done, diagnosis, score,
peers,
done, error. Shapes:
- signals: {fetchable, text_rendered, llms_txt, robots_ok, sitemap, meta}
- tasks: [{id, label}]
- task_update: {task_id, status, steps, streaming_url}
- task: {id, label, outcome, blocker, blocker_detail, answer, steps,
  expected_steps, seconds}
- diagnosis: {task_id, blocker, root_cause, fix, severity, confidence}
- score: {total, grade, success, efficiency, readability};
  agent_tax: {avg_steps, usd_per_task}
- peers: [{domain, readability}]
Build with this sample first so the page renders before the backend exists:
const SAMPLE = {score:
{total:62,grade:"C",success:41,efficiency:9,readability:12},
agent_tax:{avg_steps:17.5,usd_per_task:0.28}}.

## States you must handle
Empty (only hero), running, complete, partial failure (some tasks failed:
still
show the report), error (one friendly sentence + "Try again"), invalid URL,
leaderboard empty ("Run the benchmark to fill this table").

## Accessibility and responsiveness
All interactive elements reachable by keyboard with a visible amber focus
ring.
Contrast at least 4.5:1 for text. aria-live="polite" on the progress rail and
score. Works at 390px width: grid becomes one column, tables scroll inside
their
card, nothing scrolls the page sideways.

## Do not
- No frameworks, no build tools, no external JS besides Tailwind CDN.
- No lorem ipsum, no emoji, no stock illustrations, no fake testimonials.
- No raw JSON or stack traces on screen.
- Keep app.js under 300 lines and index.html under 250 lines; small named
render
  functions, no clever abstractions.
```

```
## Done when
A cached sample audit renders fully in under 3 seconds, a live audit shows at
least one live iframe, the score animates, diagnoses can be copied, the
leaderboard loads, and the page has no console errors at 390px and 1440px.
```

Why this prompt works: it gives the agent the story order (which is the video order), fixed function names that match the SSE wiring in Phase 5, a palette with exact values, every state including failures, and a measurable "done when." Vague UI prompts produce generic dashboards; this one produces the demo.

11. README template

The README is worth part of 40 points (GitHub plus TinyFish). Judges read the top screen and the TinyFish section; everything else is backup. Fill the angle-bracket parts with real numbers from the benchmark.

```
# AgentLighthouse

**Lighthouse tells you how your site performs for humans. AgentLighthouse
tells you
whether an AI agent can actually use it.**

![AgentLighthouse demo](docs/demo.gif)

AI agents now shop, compare and book for people. Most websites quietly break
them:
cookie walls, unlabeled buttons, CAPTCHAs, login walls. AgentLighthouse sends
real
TinyFish web agents to do real tasks on your site, watches them live, and
returns an
Agent-Readiness Score, the exact step where each agent got stuck, and a copy-
paste fix.

[Demo video](<link>) | [Post](<link>) | [Leaderboard](#benchmark-results)

## Quick start (2 minutes, no keys needed)
    git clone https://github.com/<team>/agentlighthouse && cd agentlighthouse
    pip install -r requirements.txt
    DEMO_MODE=1 uvicorn app.main:app --port 8000   # open
http://localhost:8000
DEMO_MODE replays real recorded TinyFish runs from data/cache.
For live audits: copy .env.example to .env, add TINYFISH_API_KEY and LLM
keys,
run without DEMO_MODE.
```

```
## How we use TinyFish (the core of the product)
| TinyFish API | What it does in AgentLighthouse | Code |
|---|---|---|
| Agent (run-async + runs) | Attempts 4 real tasks per site; its outcome,
step count and step trace ARE our measurement | app/tinyfish.py `run_agent`,
app/runner.py |
| Live browser stream | `streaming_url` shown live in the UI while agents
work | app/runner.py, web/app.js |
| Fetch | Reads homepage, robots.txt, llms.txt, sitemap; error codes like
`bot_blocked` become readiness signals | app/signals.py |
| Search | Finds competitors so every score is relative | app/signals.py
`peers` |
Without TinyFish there is no product: the score is literally "what happened
when a
real agent tried".

## How it works
<architecture diagram image>
1. Read the site (Fetch) 2. Plan tasks (LLM) 3. Run agents (TinyFish Agent,
live)
4. Diagnose failures (LLM reads the step trace) 5. Score + Agent Tax 6.
Compare peers (Search)

### Scoring
Score = Task success (55) + Efficiency (20) + Machine readability (25).
Agent Tax = average steps per task x $0.016 (TinyFish's per-step price).
Full formula in design.md.

## Benchmark results (<date>)
We audited <N> public sites. <X> of <N> stopped an agent before checkout.
Most common blocker: <blocker>. Agent Tax ranged from $<a> to $<b> per task.
<leaderboard table or image>
LLM diagnosis agreed with human review in <k> of 10 checked failures.

## Tech stack
Python 3.11, FastAPI, httpx, any OpenAI-compatible LLM, vanilla JS +
Tailwind.
~<lines> lines of Python. Every external call is cached (app/cache.py).

## Limitations (honest)
- Agents behave non-deterministically; scores can move a few points between
runs.
- Public pages only: no logins, no payments, no form submissions, by design.
- robots.txt parsing is simplified; English-language sites only for now.

## Roadmap
```

```
Scheduled monitoring and alerts, CI check for every deploy, auto-generated
fix PRs,
multi-language tasks, an MCP endpoint so agents can check a site before
visiting.

## Built with AI, transparently
See agents.md for a log of everything AI generated and what we verified.

## Team
<names, roles, links>
```

12. Demo video script (2 minutes 30 seconds)

Record the screen at 1080p with a clear mic; no face cam needed. Speak slowly. The first 20 seconds must show the product, not slides. Total spoken words: about 330.

Time	On screen	Voiceover
0:00 – 0:15	Live agent iframes running on a real shop, then cut to the score counting up to the grade	"This is an AI agent trying to buy something on a real website. It failed. And the site owner has no idea. AgentLighthouse finds out for them."
0:15 – 0:35	Hero section, typing a URL, clicking Run audit	"AI agents are starting to shop and book for people. But most websites were built only for human eyes. AgentLighthouse is like Google Lighthouse, but for AI agents. Paste a URL, and we send real web agents to use your site."
0:35 – 1:05	Progress rail, then the 2x2 grid of live agents	"First, TinyFish Fetch reads the site: robots.txt, llms.txt, the sitemap. Our model picks four real tasks for this kind of site. Then four TinyFish web agents go to work, live. You're watching their real browsers. This one is trying to reach checkout."
1:05 – 1:30	A task card turning red; the diagnosis card with the fix code box; click Copy	"This agent got stuck. Our AI reads the agent's step-by-step trace and explains why: here, a cookie banner covers the Add to cart button. And it writes the fix your developer can paste in today."

Time	On screen	Voiceover
1:30 – 1:50	Score card, sub-bars, Agent Tax tile	"Everything rolls up into an Agent-Readiness Score and the Agent Tax: how many steps, and how many cents, every AI agent pays to get a job done on your site. Fewer steps means more agents finish, and more sales."
1:50 – 2:10	Peers table, then the Leaderboard view	"TinyFish Search finds your competitors so the score means something. We benchmarked <N> real sites: <X> of them stopped an agent before checkout."
2:10 – 2:30	Slide: architecture diagram, then the GitHub repo page	"Under the hood, TinyFish does all the web work: Agent, live stream, Fetch and Search. The code is open, it runs in demo mode with one command, and every AI-written line is logged in agents.md. AgentLighthouse: make your website ready for its next billion visitors."

Recording checklist

- [ ] Close notifications; browser at 100% zoom; hide bookmarks bar.
- [ ] Use a cached sample for the main flow so nothing stalls; show one fresh live run for the live-agent shots.
- [ ] Cursor movements slow and deliberate. Pause one second on every key screen.
- [ ] Add captions (YouTube auto-captions, then fix the key terms: TinyFish, AgentLighthouse, Agent Tax).
- [ ] Export under 3 minutes. Upload unlisted. Test the link logged out.

13. Social media post (LinkedIn and X)

The brief says the post should show what the project does, who it helps and what makes it cool, with a video or GIF and the repo link, and it must be public. It does not need to mention the hackathon. Each team member reposts with one line of their own to multiply reach.

LinkedIn (attach the 30 to 60 second clip or the GIF)

```
We sent AI agents to shop on <N> real websites. <X> of them couldn't reach
checkout.

AI agents are starting to browse, compare and buy for people. But most
websites
were designed only for human eyes, and they quietly turn agents away: cookie
walls
over buttons, icons with no labels, login walls, CAPTCHAs. The site owner
never
finds out, because a failed agent leaves no complaint.

So we built AgentLighthouse, a Lighthouse for AI agents.

Paste a URL and it:
-> sends real TinyFish web agents to do real tasks on your site, and you
watch them live
-> pinpoints the exact step where each agent got stuck
-> uses AI to explain why, and writes the fix your developer can paste in
-> gives you an Agent-Readiness Score and your "Agent Tax": what every AI
agent
   pays, in steps and cents, to get things done on your site

What we found across <N> sites:
- Most common blocker: <blocker>
- Agent Tax ranged from $<a> to $<b> per task
- <one surprising finding>

Built with TinyFish Agent, Fetch and Search, FastAPI and an LLM diagnosis
layer.
Open source: <repo link>
Demo: <video link>

If you run an online store or a SaaS, comment your URL and we'll audit it.

#AIagents #AgenticCommerce #WebDev #OpenSource #AI
```

X thread (4 posts)

```
1/ We sent AI agents to shop on <N> real websites. <X> couldn't reach
checkout.
Meet AgentLighthouse: a Lighthouse for AI agents. [GIF]

2/ Paste a URL. Real web agents (TinyFish) try real tasks on your site while
you
watch. When one gets stuck, AI reads its step trace and writes the fix.

3/ You get an Agent-Readiness Score and your "Agent Tax": what each AI agent
pays,
in steps and cents, per task on your site. Ours ranged $<a> to $<b>.

4/ Open source, runs in demo mode with one command: <repo link>
Demo video: <link>
```

Post checklist: visual attached; repo link works; post set to public; checked in a logged-out window; link pasted into Unstop.

14. PPT deck content, slide by slide

Twelve slides that follow the judging rubric in order, so a judge can score as you present. Use the same dark-and-amber look as the UI. One idea per slide, big type, screenshots over bullet text. The deck is for the finale and for any screening that asks for one; the video does not need it.

1. **Title**
   - On slide: "AgentLighthouse", the line "Lighthouse for AI agents", team name, product screenshot of the score card.
   - Say: "We test whether AI agents can actually use your website."
2. **The shift**
   - On slide: "AI agents could handle 15 to 25% of US e-commerce by 2030" (Bain) and "$300 to 500 billion." One image: an agent cursor on a checkout page.
   - Say: "Your next customer might not be a person."
3. **The problem**
   - On slide: four icons for cookie wall, unlabeled button, CAPTCHA, login wall. Caption: "Invisible to humans. Fatal to agents. Never reported."
   - Say: "A failed agent leaves no complaint and no analytics trail. Owners are blind."
4. **Why existing tools miss it**
   - On slide: two columns. "Static checkers (Cloudflare, Frase): read robots.txt, llms.txt, schema." versus "AgentLighthouse: sends a real agent to do the job and watches it." Analogy line: "Spec sheet vs. crash test."

5. **The product (demo slide)**
   - On slide: the 4 live-agent grid screenshot, or play the 30-second clip.
   - Say the three-step flow: paste URL, watch agents live, get score, diagnosis and fix.
6. **How TinyFish powers it (20 points live here)**
   - On slide: the architecture diagram from section 4 with the 3 TinyFish steps highlighted, plus a 4-row table: Agent, live stream, Fetch, Search and what each does.
   - Say: "Remove TinyFish and there is no product. The score is literally what happened when a real agent tried."
7. **The AI/ML inside**
   - On slide: site classification, task planning, step-trace diagnosis with blocker taxonomy, fix generation; the scoring formula as one line; "Diagnosis agreed with human review in k of 10 cases."
8. **Results: the benchmark**
   - On slide: leaderboard table of N real sites with grades; 3 headline numbers (sites that failed checkout, most common blocker, Agent Tax range).
   - Say the single most surprising finding.
9. **The Agent Tax**
   - On slide: two bars, Site A 9 steps vs Site B 31 steps or failed, with cents per task.
   - Say: "Every extra step is a cost every agent pays, and a sale at risk."
10. **Business model**
    - On slide: the 4 price tiers with gross margins from section 15, and the unit-cost line "about 12 steps x $0.016 = $0.19 per task."
11. **Market and go-to-market**
    - On slide: 6.9M Shopify stores worldwide, about 60,000 in India, year-1 target 600 customers; GTM steps: leaderboard, Shopify app, agencies.
12. **Roadmap and ask**
    - On slide: monitoring and alerts, CI check per deploy, auto-fix pull requests, an MCP endpoint agents call before visiting a site. Team photos and roles. Repo QR code.
    - Close: "Make your website ready for its next billion visitors."

Deck rules: maximum 25 words of text per slide (except the pricing table); every number on a slide has its source in small type; rehearse to 5 minutes flat with the clicker.

15. Revenue model and market research

We sell subscriptions priced on the number of agent tasks run per month, because agent steps are our main cost. Every tier keeps a gross margin of about 65% or more, even at TinyFish's pay-as-you-go price, and margins rise on TinyFish's volume plans.

Unit economics (the number judges ask about)

- TinyFish Agent costs about $0.016 per step; Search and Fetch are free (TinyFish). Volume plans drop overage to $0.012 per step (TinyFish pricing).
- Our planning assumption: **one task averages 12 agent steps, so about $0.19 per task.** Replace 12 with the real average from tonight's benchmark before the deck goes out.
- LLM diagnosis and hosting add well under $0.01 per task, so we ignore them in the table.
- Prices in rupees, with US dollars at an assumed ₹85 per dollar.

Plan	Price per month	What's included	Agent tasks per month	Our cost per month	Gross margin
Free	₹0	1 audit, 4 tasks, public report link	4	about $0.77	Lead generation (capped)
Starter	₹1,499 (about $18)	Weekly audit, 6 tasks each, fix list, email alert when score drops	24	about $4.60	about 74%
Growth	₹4,999 (about $59)	Daily check on 3 key journeys, monthly competitor audits, CI webhook	102	about $19.60	about 67%
Agency	₹14,999 (about $176)	20 client sites audited weekly, white-label reports	320	about $61	about 65%
Enterprise	from ₹75,000	A check on every deploy, custom journeys, SSO, SLA	Custom	Custom	Priced to stay above 65%

Why these prices are justified

- **Cheaper than what teams already pay for human-visibility tools.** Semrush's entry plan is $139 a month or more (seo.com). Our Growth plan is under half of that, for a new channel Semrush doesn't test.
- **More expensive than static checkers, for a reason.** Static audits are nearly free; one marketplace tool charges from $5 per 1,000 static audits (Apify). We run real browsers doing real tasks, which costs real compute, and that's the evidence a static check cannot give. Our free tier competes with free static tools.
- **Value anchor for a store owner:** if one agent-driven order a week is lost to a broken flow, Starter pays for itself with a single recovered sale.

Other revenue lines

1. **Pay-as-you-go API for platforms and agencies:** $1.50 per 4-task audit (cost about $0.77, about 49% margin).
2. **Fix partner referrals:** 15 to 20% referral fee when a store hires a listed agency to apply fixes.
3. **Sponsored research:** a quarterly "State of the Agentic Web in India" report built from leaderboard data.

Market sizing

Level	Who	Size	Source or assumption
Demand driver	AI agents' share of US e-commerce by 2030	15 to 25%, $300 to 500 billion	Bain
Total addressable (one platform only)	Live Shopify stores worldwide	about 6.9 million	Exploding Topics, Feb 2026
Beachhead	Active Shopify stores in India	about 60,000	StoreCensus, Sep 2026
Year-1 target (assumption)	1% of Indian Shopify stores at a blended ₹2,500 a month	600 customers, about ₹1.8 crore a year	Our assumption; plus agencies and SaaS on top
Global upside (assumption)	1% of worldwide Shopify stores at $30 a month	about 69,000 customers, about $25 million a year	Our assumption; excludes SaaS, travel and other platforms

Go-to-market in three steps

1. **Public leaderboard plus "comment your URL" audits on LinkedIn.** Every free report is shareable and links back.
2. **Shopify App Store listing** with a one-click weekly audit (Starter plan).
3. **Agencies as the channel:** they already sell SEO audits; we give them a new line item that is easy to sell, and white-label reports.

16. Judge Q&A prep

The pitch owner answers first; the person who built that part adds one sentence only if needed. Every answer is under 30 seconds and ends with a fact. Drill these out loud twice before the finale.

Likely question	Answer
"Cloudflare and others already score agent readiness. What's new?"	"They read files. We send an agent to do the job. A site can have a perfect llms.txt and still trap an agent behind a cookie banner on checkout. Our tasks and live traces catch that; static checks can't."
"Agents are non-deterministic. Isn't the score noisy?"	"Yes, a few points between runs, and we say so in the README. The blockers are not noisy: a cookie wall over Add to cart fails every time. For paid plans we'd run each task twice and report the median."
"Isn't this just testing TinyFish's agent rather than the site?"	"We hold the agent constant and vary the site, like a crash-test dummy. Same agent, same task, very different results across sites. That difference is the site."
"Why would a store owner pay?"	"Because agent-driven orders are a growing channel they can't see. One lost order a week covers the Starter plan, and the report gives fixes, not just a number."
"How do you make money at TinyFish's prices?"	"About 12 steps per task at $0.016 is about $0.19. Starter runs 24 tasks for ₹1,499, so about 74% gross margin, and volume pricing improves it."
"Is it legal or ethical to run agents on other people's sites?"	"Agents only browse public pages like a shopper: no login, no forms, no payment, at most 4 runs per audit. The leaderboard is dated and labelled as a usability snapshot, not a verdict on the company."
"What's the ML here, beyond calling an LLM?"	"Classification of site type, planning tasks, structured diagnosis into a fixed taxonomy from step traces, and we measured it: the diagnosis matched human review in k of 10 cases. The scoring is deterministic and unit-tested."
"What happens when sites fix things?"	"That's the business: scheduled re-audits show the score going up and alert when a deploy breaks an agent journey."

Likely question	Answer
"What would you build in the 24-hour finale?"	"Scheduled monitoring with alerts, plus a CI check that fails a deploy if an agent can no longer reach checkout."
"How much of this did AI write?"	"A lot of the boilerplate, and it's all logged in agents.md with what we verified. The problem, the metric, the scoring, the taxonomy and the benchmark review were ours."

17. Risks and fallbacks

Decide fallbacks now, not at 22:30. If a risk fires, the owner posts `BLOCKED` and switches to the fallback within 20 minutes.

Risk	Early sign	Fallback	Owner
TinyFish credits run out	Wallet balance under $3, or `BILLING_FAILURE` errors	Stop live runs; everything else uses the cache. Top up the $10 minimum if a live demo is essential	A
A site blocks the agent (CAPTCHA, bot wall)	`captcha_or_bot_block` on every task	That is a valid finding, so keep it in the report. Swap the site out of the benchmark only if every task fails	B
Agent runs are slow (over 2 minutes each)	Audit takes over 6 minutes	Lower `max_duration_seconds` to 90; benchmark with 3 tasks instead of 4	A
`streaming_url` iframe won't load	Blank iframe in the UI	Show the step counter and last action text; use the recorded `video_url` in the video	C
`output_schema` rejected (HTTP 400)	Error mentions a keyword	Remove the named keyword; the docs list `additionalProperties`, `const` and `oneOf` as rejected	A

Risk	Early sign	Fallback	Owner
LLM returns bad JSON or generic advice	Diagnosis says "improve UX"	Tighten the SYSTEM prompt with one worked example; drop diagnoses with confidence under 0.4 from the UI	B
UI not ready by 21:00	Gate P5 failing	Ship the report page only (score, tasks, diagnoses), without the leaderboard; record the video from that	C
Wi-Fi fails during recording or the finale	Requests hang	`DEMO_MODE=1` replays everything from cache	Everyone
Repo link private or broken at submission	Logged-out check fails	Fix visibility first; this check happens at 22:15, not 23:55	D
Running out of time	Any phase over by 30+ minutes	Cut in this order: peers, leaderboard view, sample buttons. Never cut: live agent view, score, diagnosis, README	A

18. Final submission checklist and finale prep

Submission checklist (D ticks each box; nothing is assumed)

- [ ] Exact Unstop cutoff time confirmed; alarm set 60 minutes before.
- [ ] Repo is public; README opens with the GIF and the one-liner.
- [ ] README has the "How we use TinyFish" table and links to the code.
- [ ] `DEMO_MODE=1` quick start tested from a clean clone on a second laptop.
- [ ] `design.md` and `agents.md` are in the repo root; agents.md has a log row per phase.
- [ ] At least 15 commits with `phase N:` messages, spread across the day.
- [ ] `.env` is not in the repo; no keys anywhere in the history (`git log -p | grep -i api_key` shows nothing real).
- [ ] Demo video is 2 to 3 minutes, shows the live TinyFish agent, plays logged out.
- [ ] Social post is public, has the clip or GIF and the repo link, and opens logged out.
- [ ] Unstop form submitted with repo, video and post links; confirmation screenshot saved.

If shortlisted: finale prep (31 October to 1 November)

The finale is a 24-hour build-and-evaluate round followed by finals in front of the jury. Arrive with a plan, not just a demo.

1. **Pre-decide the 24-hour build:** scheduled monitoring with score-drop alerts, and a CI check (a GitHub Action) that fails a deploy when an agent can no longer reach checkout. Both reuse the existing pipeline.
2. **Pre-write the design.md section** for those features before you travel, and get the team to agree on it.
3. **Refresh the benchmark** a few days before with a larger set of sites, and bring before-and-after examples if any site fixed something.
4. **Rehearse the 5-minute pitch** with the deck in section 14 and the Q&A in section 16, with a timer, at least three times.
5. **Carry the demo offline:** the cached `DEMO_MODE` build on two laptops, the video on a USB drive, and a phone hotspot.
6. **Bring credits:** top up TinyFish in advance so live runs at the finale never stall.
7. **Network with intent:** have a 30-second version of the pitch and offer to audit any site live for people you meet; it's the best demo you have.

Sources

All pages below were opened on 20 October 2026.

- TinyFish: For Coding Agents (endpoints, SDKs, latency, gotchas)
- TinyFish: Agent Endpoints (run, run-async, run-sse, request options)
- TinyFish: Runs (run object, steps, statuses, errors)
- TinyFish: Structured Output (output_schema rules)
- TinyFish: Search API Reference
- TinyFish: Fetch API Reference
- TinyFish: Pricing and TinyFish blog on per-step pricing
- Bain: 2030 forecast for agentic AI in US retail
- Search Engine Journal: Cloudflare's Agent Readiness Score
- Frase: Agent Readiness Checker
- Chat Thing: Agent Readiness Checker
- Apify: AI Agent Readiness Audit
- seo.com: Semrush pricing
- Exploding Topics: number of Shopify stores
- StoreCensus: Shopify stores in India